

Registro de actividades realizadas

Formato: [Actividad]: [Fecha] - [Integrantes] - [Duración] - [Commit]

Sprint Planning y primeros avances: 15/04/2026 - Atrio, Robayna - 2h - 59d7656

Definición de proceso de ingeniería: 19/04/2026 - Viera - 45min - 7c798ae

Modelo de calidad, deuda técnica, revisión de docs y búsqueda de issues:

19/04/2026 - Atrio - 2h - 59d7656

Levantar obligatorio, leer la letra y probar funcionalidades: 20/04/2026 - Robayna - 3h

Revision final: 20/04/2026 - Atrio, Robayna, Viera - 1h

Retrospectiva: 20/04/2026 - Atrio, Robayna, Viera - 1h -

Evidencia de resultados de la entrega

Definición de marco de gestión basado en Kanban

Se creó un tablero Kanban en Github projects con columnas: ToDo, InProgress, In Review, Done. Se le asignó a las columnas InProgress e InReview un WIP de 3 debido a que somos 3 desarrolladores. <https://github.com/orgs/IngSoft-ISA2-2026-1/projects/31>

Roles y ceremonias: Se asignaron los siguientes roles: PO - Xoan Atrio, Encargado de ceremonias (similar a Scrum Master) - Lautaro Robayna, Desarrollo y Testeo - los 3. Los roles rotarán en cada iteración.

Se pensaron las siguientes ceremonias por iteración: Sprint planning al inicio(1h), una daily en la mitad(10 min), una review que sería una reunión con el PO(30 min) y una retrospective al finalizar(1h) que se hará con metro retro.

Definition of Ready: Escritura de US (como, quiero, para), criterios de aceptación, tiempo estimado y escenarios de prueba BDD, todo hecho y/o revisado por al menos 2 integrantes.

Definition of Done: Una feature será marcada como finalizada cuando pase todos los tests y sea revisada por el PO de esa iteración.

Definición de issues: Una issue será cualquier tipo de error o mejora que se pueda realizar sobre el proyecto actual y deberán tener labels asignados entre los siguientes: feature, bug, docs, infra, telemetry, refactor. También deberán tener un título, una descripción. Los labels se pueden ver configurados en el siguiente link:

<https://github.com/IngSoft-ISA2-2026-1/proyecto-m7b-atrrio-robayna-viera/labels>

Definición de un primer proceso de ingeniería

Se define el siguiente proceso de ingeniería inicial para la primera entrega. El mismo será revisado y ajustado en cada entrega posterior según corresponda (por ejemplo, incorporación de TDD en la Entrega 2 y BDD en la Entrega 3).

Flujo de trabajo: Toda funcionalidad nueva, bug, tarea de infraestructura o documentación se crea como issue en GitHub Projects con título, descripción, label (feature, bug, docs, infra o telemetry) y responsable asignado. Al crearse se ubica en ToDo. El integrante que toma un issue lo mueve a InProgress (respetando el WIP de 3), crea una rama desde main con la nomenclatura tipo/numero-descripcion (ej: feature/12-login-usuarios) y desarrolla el cambio. Al finalizar abre un Pull Request hacia main que referencia al issue (Closes #N) y lo mueve a InReview. El PR debe pasar los checks automáticos del pipeline (build, tests, linter) y ser aprobado por al menos otro integrante. Tras el merge, la rama se elimina, el issue pasa a Done y se cierra. Al cierre de la etapa se agrega el tag Entrega N sobre main.

Estándares de commits: Los mensajes siguen el formato tipo(alcance): descripción (ej: feat(auth): agrega endpoint login). A partir de la Entrega 2 se suman los rótulos requeridos por la rúbrica: [Green], [Refactor], [TDD-GREEN], [TDD-REFACTOR], [BDD-GREEN], [BDD-REFACTOR].

Estándares de código:

- Se aplica linter y formateador en ambos lados de la aplicación (ESLint + Prettier en Angular, analizadores de .NET en el back). No se mergea código que no pase el linter.
- Se respetan los criterios de clean code: nombres significativos, sin magic numbers ni strings, funciones cortas con responsabilidad única.
- Todo cambio en el backend debe estar respaldado por pruebas unitarias. El objetivo de cobertura es 100% según el modelo de calidad definido.
- No se versionan secretos ni datos sensibles. Las configuraciones se manejan por variables de entorno o archivos ignorados por git.

Revisión y gates: Todo cambio a main pasa obligatoriamente por Pull Request. El PR debe tener al menos un reviewer distinto del autor y debe pasar todos los checks automáticos del pipeline de CI (GitHub Actions) antes de poder mergearse.

Roles durante el proceso: Los tres integrantes actúan como desarrollador y tester en todas las entregas. El Product Owner de la iteración valida que el trabajo cumpla con los criterios de aceptación antes de que un issue pase a Done. El Encargado de ceremonias garantiza que se estén cumpliendo las prácticas definidas (Definition of Ready, Definition of Done, revisión por pares). Los roles de PO y Encargado de ceremonias rotan en cada entrega.

Herramientas: Gestión de issues y tablero Kanban en GitHub Projects. Versionado y revisión en GitHub. CI/CD en GitHub Actions (se configura en detalle en la Entrega 2). Testing unitario con el framework de .NET en el backend y Jasmine/Karma en el frontend.

Evolución del proceso: Este proceso es un punto de partida y se irá adaptando en las siguientes entregas. En cada informe de avance se reportará si hubo cambios al proceso de ingeniería o si se mantiene estable.

Configuración del repositorio en GitHub

Ramas: Se adoptó Trunk Based Development como estrategia de branching. La estructura de directorios, flujo de ramas y estándares de nomenclatura se detallan en el proceso de ingeniería

Carpetas: Raíz: Carpetas principales separadas por responsabilidades. Material: Código fuente de la aplicación y letra. Código fuente separado por backend y frontend. Documentación: Informe de cada entrega. Se añadirán más carpetas cuando se presente una nueva responsabilidad.

Commits: Los estándares de nomenclatura son los definidos en el proceso de ingeniería.

Análisis de deuda técnica y gestión de calidad

En este proyecto se definió un modelo de calidad basado en los atributos de calidad que pueden ser priorizados en un proyecto universitario.

Mantenibilidad: Que cumpla con todos los criterios de clean code relevantes (desde magic numbers o strings hasta comentarios y formato). Cumplimiento de la guía de estilos utilizada con formateador automático. Verificación de complejidad de métodos, sin métodos más complejos de lo necesario. Tanto en escala $O(n)$ como en tamaño y complejidad de entendimiento. Porcentaje de entradas inválidas que el sistema maneja sin romperse debe de ser del 100% (porcentaje de excepciones sobre total de caminos inválidos).

Portabilidad: La aplicación debe ser 100% portable en sistemas virtuales como docker. Uso correcto de archivos de configuración y variables de entorno. Se debe poder ejecutar el sistema solo con descargarlo y ejecutar un comando.

Testeabilidad: Se debe tener un pipeline CI/CD para tests. 100% de coverage en todos modulos. Cubrir todos los caminos de éxito y todos aquellos caminos con fallos que sean relevantes. Pruebas unitarias con cobertura total más pruebas de integración requeridas.

Desplegabilidad: Se debe tener un pipeline CI/CD para el despliegue, totalmente automatizado. Desde un push a main se debe tener un despliegue en menos de 5 minutos. Se debe contar con versionado con posibilidad de rollback.

Seguridad: El código debe cumplir con medidas de seguridad básica (integrar análisis de vulnerabilidades en el código automatizado). Principalmente cumplir con no guardar datos sensibles en bruto ni retornar más información de la necesaria. Verificar permisos.

Observabilidad y Monitoreo: Se cuenta con logs estructurados para todas las funcionalidades principales. Se utilizan herramientas para monitorear el estado del sistema en tiempo real. Se envían alertas automatizadas bajo ciertas condiciones del sistema.

Se hará el análisis en base al modelo de calidad creando issues donde no se cumpla. Principales issues detectadas en seguridad, despleabilidad, testeabilidad y mantenibilidad.
<https://github.com/IngSoft-ISA2-2026-1/proyecto-m7b-atrío-robayna-viera/issues>

Mejoras en la calidad y evidencia de revisión con el PO

Con base en todos los estándares y procesos planteados se espera que al resolver los problemas detectados y se continúe con un proceso de mejora continua se vean resultados directos en la calidad del proyecto tanto desde el punto de vista del cliente como de los desarrolladores del mismo. En esta entrega no se realizaron cambios directos a la aplicación por lo que la revisión con el PO fue dirigida hacia la documentación y esta primera etapa de planificación, se tuvieron en cuenta errores sencillos como información duplicada en varias secciones. Pero el enfoque de esta revisión fue principalmente sobre el primer proceso de ingeniería y el modelo de calidad diseñado, postergando aspectos como “Performance”, definiendo métricas más precisas como el porcentaje de entradas inválidas como excepciones, clarificando estándares de código y seleccionando y unificando issues.

Revisión y acciones de mejora

Para la retrospectiva se utilizó la estrategia “El bueno, el malo y el feo” en la que destacamos que salió bien, que no nos convenció, que podría haber salido mejor y qué objetivos tenemos como equipo. En resumen nos gustó la comunicación asincrónica por mensajes y las definiciones que utilizamos, pero nos pareció que deberíamos habernos organizado más para tener reuniones frecuentes y realizar el trabajo de forma más continua durante la iteración y no sobrecargar el final. <https://ludi.co/BOR2WHWUVZS7>